

TexeraAgent: An AI-Agent for Data Science Using Dataflows

Jiadong Bai
University of California, Irvine
Irvine, California, USA
jiadongb@ics.uci.edu

Yicong Huang
University of Massachusetts Amherst
Amherst, Massachusetts, USA
yiconghuang@cs.umass.edu

Chen Li
University of California, Irvine
Irvine, California, USA
chenli@ics.uci.edu

ABSTRACT

Recent advances in large language models (LLMs) have made human-agent collaboration a promising and powerful paradigm for data science. Among existing LLM-based solutions, conversational analytics is convenient but makes it difficult for users to inspect intermediate steps and verify errors. Script-based ReAct improves transparency by grounding interaction in coding scripts, but still inherits the complexity and low-level details of programming code. In this paper, we present a novel ReAct-based AI agent called TexeraAgent in the Apache Texera system. This agent is built on the idea of using dataflow as the abstraction for human-agent collaboration. Such an abstraction is especially friendly for LLMs to do their internal reasoning for data science tasks. We will show the benefits of TexeraAgent, including how an analyst easily understands the agent’s behaviors through an intuitive dataflow-based interface, how the analyst efficiently inspects the agent’s past actions to provide feedback, and how the agent can finish a task with high accuracy and low cost.

CCS CONCEPTS

• **Information systems** → **Data analytics**; **Computing platforms**; • **Software and its engineering** → **Data flow architectures**.

KEYWORDS

Human-AI collaboration, Data Science, Agent, LLM, Dataflow

1 INTRODUCTION

Recent advances in large language models (LLMs) have made human-agent collaboration increasingly promising for data science. A common approach is conversational analytics, where human analysts use natural language to ask agents to explore data, generate plots, and summarize findings, using LLM services such as ChatGPT and Claude. While convenient, its interface typically limits human analysts to high-level supervision [4] and provides little visibility into the underlying data or intermediate steps. As a result, hallucinations and analysis errors are difficult to avoid and verify [5], and the human-agent interaction becomes hard to manage for complex, iterative, and multi-step tasks. Recent systems have shifted toward a “Script ReAct” paradigm, e.g., Databricks Genie Code [3], in which a human analyst and the agent collaborate on executable code and notebooks. As shown in the top part of Figure 1, an analyst in such a system describes the task in natural language, and the agent generates executable scripts (often in the form of notebook cells) that serve as the shared artifact for collaboration. By grounding interaction in executable artifacts, Script ReAct offers greater transparency and control than conversational analytics [12].

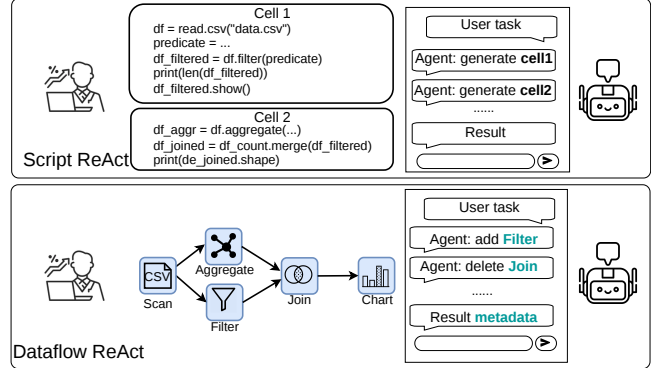


Figure 1: Script ReAct versus dataflow ReAct (TexeraAgent, this work) on data science tasks.

While more powerful, Script ReAct still inherits the following limitations of a programming interface. (1) *Human barrier and cognitive overhead*. Programming languages are not an ideal medium for analysts: they raise the entry barrier for non-programmers, and even experienced programmers must spend effort reading code before they can verify or revise the intended analysis. (2) *Weak artifact traceability*. While code itself can be versioned, its outputs often live in ephemeral artifacts (in-memory states, temporary files, visualizations, console logs), making it hard to track dependencies and maintain provenance across iterations. (3) *Noisy agent context*. Low-level code details (syntax, variables, control flow) inflate token cost, degrade grounding over long interactions, and increase the risk of hallucinated steps.

In this paper, we present a novel idea to leverage LLMs in data science based on the following insight. Before the emergence of LLM agents, the data management and workflow communities already developed dataflow systems as an effective *abstraction* for complex data analytics. In these systems, computation is represented as a graph of operators connected by data dependencies (often as a directed acyclic graph or DAG) [1, 9]. This abstraction is appealing because it naturally exposes the semantic structure of an analysis while suppressing low-level implementation details [1, 16]. Even relational DBMS systems expose execution plans through interfaces such as EXPLAIN in PostgreSQL. Such an abstraction can be especially friendly for LLMs to do their internal reasoning on data tasks.

Based on this insight, our idea is to use dataflow as the primary abstraction for human-agent collaboration in data science tasks, as shown in the bottom part of Figure 1, which is referred to as *Dataflow ReAct*. Compared to Script ReAct, we replace code with dataflow as the collaboration medium, so that both the analyst and the agent reason over a higher-level analytical structure

rather than low-level programming details. We present *TexeraAgent*, an LLM-based agent on top of the Apache *Texera*¹ system. The agent can collaborate with analysts through interactive dataflows rather than through script-based notebooks. Specifically, *TexeraAgent* consists of an interactive dataflow UI, a chatbox, a stateless dataflow-execution engine, and a catalog-governed lakehouse. The agent understands an analyst’s requests, constructs corresponding dataflows, presents them to the analyst for inspection, and submits them to the execution engine. It captures dataflow-aware contexts and keeps track of context changes through governance versions.

In the paper, we show how *TexeraAgent* enables human analysts to better trace its analytical process, inspect the rationale behind intermediate decisions, and intervene with effective feedback during task execution. Our preliminary benchmark further highlights that *TexeraAgent* achieves an accuracy of 79% for evaluated data science tasks, whereas a script-based ReAct baseline achieves an accuracy of 68%.

2 SYSTEM OVERVIEW

Apache *Texera*. It is an open-source system for human-AI collaborative data science using visual workflows. It enables analysts to construct, execute, and refine data analysis tasks through an intuitive GUI, assisted by AI agents that understand natural-language instructions. *Texera* is well suited for a wide range of applications, including “AI for Science,” by making advanced AI and data science capabilities accessible to a broader community. It can run on a laptop for local use or be deployed in the cloud to support scalable processing of large datasets. Started in 2016 and now under Apache incubation, the system supports parallel execution of data science and AI/ML workflows, real-time collaboration, Python/Java UDFs, and decoupled compute-storage for cloud deployment. More details are available at the project site [2].

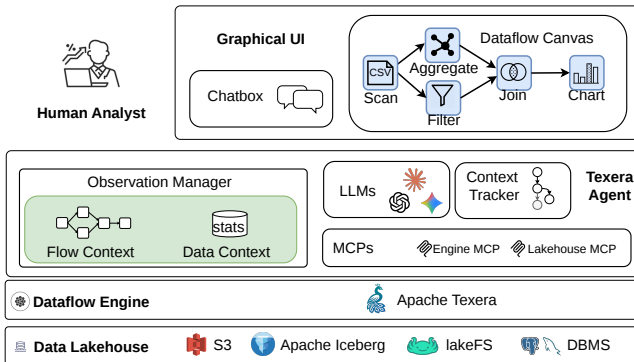


Figure 2: *TexeraAgent* system architecture.

Figure 2 illustrates the architecture of the system with *TexeraAgent*. It consists of four layers: a graphical user interface (GUI), the LLM-based *TexeraAgent*, a dataflow engine, and a data lakehouse. At the top, the GUI includes a chatbox for analysts to communicate with *TexeraAgent* in natural language, and a canvas for visualizing a dataflow. The canvas exposes the high-level structure of the dataflow, while optionally allowing human analysts to inspect operator-level details such as code and property values (e.g.,

a threshold of a filter operator). At the bottom, the dataflow engine executes a dataflow, while the data lakehouse stores data such as CSV files, generated artifacts, execution output, and runtime logs.

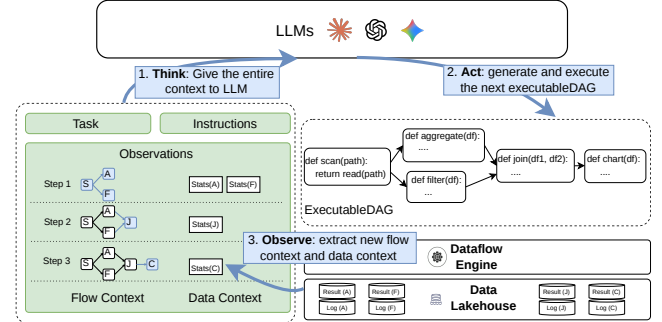


Figure 3: The “Think, Act, and Observe” loop for dataflow ReAct used by *TexeraAgent*.

At the core of the system is the *TexeraAgent*, which coordinates reasoning, execution, and observing. It connects to one or more LLMs, and interacts with the dataflow engine and data lakehouse through Model Context Protocols (MCPs). In particular, the engine MCP allows the agent to edit and inspect the dataflow and submit the dataflow for execution, while the lakehouse MCP allows the agent to retrieve metadata and artifacts from the storage. Within the agent, an *Observation Manager* maintains two dataflow-oriented context abstractions, namely “Flow Context” and “Data Context.” A “Context Tracker” does version control of these contexts together with other generated artifacts across iterations.

Human-Agent Collaboration with Dataflow ReAct Loop. When the analyst provides a task through the chatbox, the agent starts the dataflow ReAct loop, as illustrated in Figure 3. The agent takes multiple steps to complete the task, and each step contains *Think*, *Act*, and *Observe*. During thinking, the agent packages the task, the latest observations, together with instructions on how to use MCPs as a context, and sends it to an LLM to reason about the next move. The LLM generates changes to the previous dataflow. The agent acts to update the dataflow shown in the UI and submits the new *ExecutableDAG* to the engine for execution. Once the execution completes, the agent collects observations on both the new dataflow and the execution output and saves them in the data lakehouse. The agent repeats this Think-Act-Observe loop until it has made sufficient progress on the task. Throughout the process, the analyst remains in the loop and can monitor agent-driven changes to the dataflow, inspect execution results and runtime information, interrupt execution, revise the dataflow directly, or provide feedback to the agent to redirect subsequent reasoning and actions.

Observations Designed for Dataflow ReAct. As illustrated in Figure 3, the *Observe* step is designed not to feed raw code or low-level runtime states to *TexeraAgent*, but to extract two high-level, dataflow-oriented observations, namely *Flow Context* and *Data Context*. The Flow Context summarizes the structure and semantics of the analysis as a dataflow, including operators, dependencies, connectivity, lineage, and stage-level semantic roles. In Figure 3, after step 2, the agent extracts the newly added operator *Chart* together with its incoming edge from *Join* as part of the update to Flow Context in step 3. The Data Context summarizes properties

¹Currently under Apache incubation.

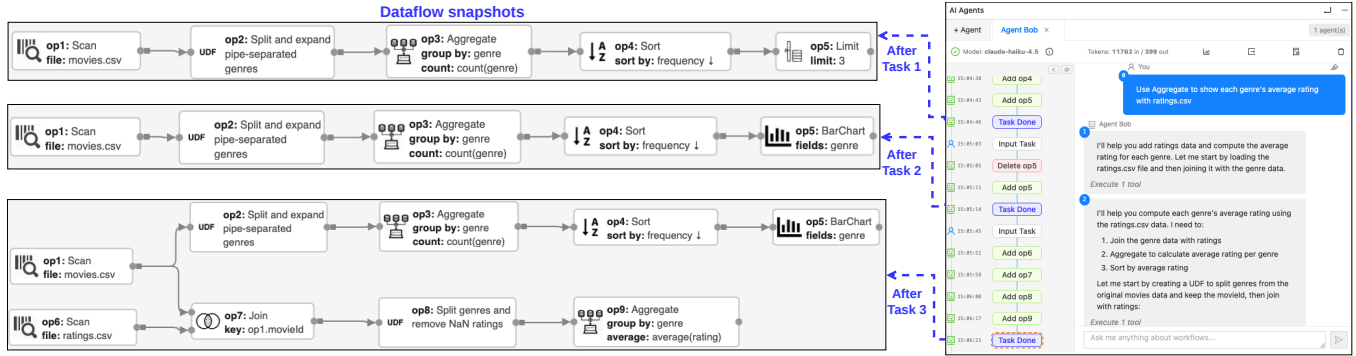


Figure 4: The TexeraAgent interface across three tasks. The dataflow canvas (left) shows the dataflow evolving in real time with operator-level semantics. The chatbox (right) displays the agent’s reasoning and keeps track of context versions.

of the data and generated artifacts, including schema, metadata, basic statistics, distributions, and profiles of intermediate and final results. In Figure 3, after step 2, the agent fetches the schema and statistics of the newly produced *Result(C)* as part of the update to the Data Context in step 3.

Context Tracking with Governance Versions. To support iterative collaboration, TexeraAgent includes a Context Tracker that controls versions of artifacts and contexts throughout the interaction. The tracker records both the artifact evolution and the dataflow evolution, including datasets, visualizations, and other execution output, as well as added, removed, or modified operators and edges in the dataflow. It also controls versions of the associated Flow Context and Data Context, allowing the agent to reason over the current state while retaining access to prior states whenever needed. This governance improves provenance and reproducibility by maintaining explicit links among dataflow versions, execution runs, and derived artifacts, allowing both the human analyst and the agent to connect each result to the dataflow and execution context that produced it. It enables analysts to comment on prior states and compare alternatives, and allows the agent to interpret feedback in the corresponding historical context and refine or branch analyses from earlier versions.

3 EXAMPLE SCENARIOS

We will use a data science example to explain TexeraAgent, where an analyst named Alice wants to study how genres relate to movie ratings stored in two files, *movies.csv* and *ratings.csv*, which share a movie-ID column. We simplify the schemas and tasks for illustration purposes.

3.1 Analyst Monitoring and Understanding Agent’s Behavior through a Dataflow

Initially, Alice wants to gain insights about genres of the movies, and sends the following task to the agent:

Task 1: “Compute the top three genres of *movies.csv*.”

While *TexeraAgent* iterates through the Dataflow ReAct loop, Alice can follow the agent’s progress in real time through the canvas, which renders the latest dataflow after each iteration. Figure 4 shows the dataflow snapshot after this task is finished by the agent. The agent reads *movies.csv* with a scan operator, uses a Python

UDF to split nested genres into separate records, then chains aggregate, sort, and limit operators to count, rank, and keep the top three genres. With the dataflow and its execution displayed on the canvas, Alice can easily follow the process and check the result.

Next, Alice wants to visualize the genre distribution and sends the following task:

Task 2: “Show the genre distribution instead.”

The agent starts a new ReAct process. From the interface Alice can see that it resumes from task 1’s context, with the version timeline clearly showing continuity between these two tasks. The second dataflow snapshot in Figure 4 shows that the agent takes two ReAct steps for this task. First, the agent understands that the analyst is looking for a full distribution of all the genres rather than the top three. As a result, it removes the limit operator. Second, the agent adds a bar chart operator to display the genre distribution. With the help of the agent, Alice has a better understanding of the genres. Next, she sends a new task to relate ratings to genres:

Task 3: “Calculate each genre’s average rating with *ratings.csv*.”

The rest of Figure 4 shows the dataflow snapshot and the chatbox after the agent finishes task 3. Alice can see that the agent adds a scan operator to load *ratings.csv* as a table and introduces an operator to join the table with the movies table on the column “movie ID.” By checking the statistics of the result, the agent identifies that there are too many records containing “NaN” values in the output. Thus, it connects another Python UDF to remove those records. After that, it connects an aggregate operator to calculate the average rating per genre.

3.2 User Inspecting Agent’s Past Actions to Provide Feedback

Alice notices that the final output contains many “NaN” values, and wants to confirm if these values are produced by join or are from the original ratings file. She inspects the results from the join operator. From the statistics of the rating column, she notices about 20% of the records contain “NaN.” Then she traces the dataflow downstream and finds that the UDF operator drops those records. Alice prefers to replace these values with zero so that genres’ ratings are not inflated. To do so, she clicks on the past action that introduced the UDF operator, and sends the following to the agent:

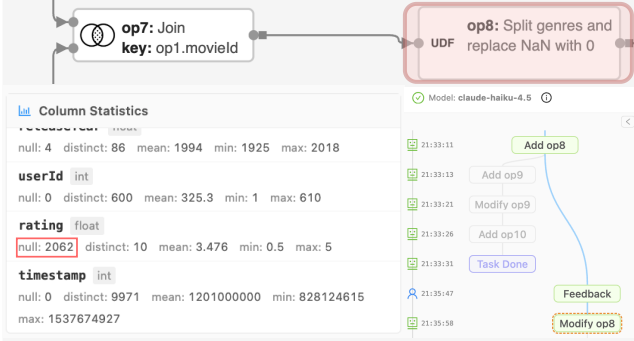


Figure 5: Alice inspects the statistics of the join operator’s result, identifies several NaN ratings, and provides feedback to the UDF operator. *TexeraAgent* branches out to revise the operator.

Feedback: “Treat all the NaN ratings as 0 instead.”

After receiving the feedback, the agent recognizes it as a response to a past action. It uses the Context Tracker to branch from the version after the UDF operator was introduced, re-assembling a detached context (dataflow structure, execution results, and observations from that version) and combining it with Alice’s feedback to start a new ReAct loop. After the thinking process, the LLM returns an action that modifies the UDF’s code to replace “NaN” values with 0. Alice sees that a new action step branches out from the past version, as shown in Figure 5. *TexeraAgent* builds future ReAct steps on this new branch. All actions on the original branch are still kept in case Alice wants to investigate or add feedback from there again. After a few more ReAct steps, the agent finishes a new analysis dataflow, and Alice sees that the genres’ ratings drop by 0.6.

3.3 Higher Accuracy and Lower Cost

The dataflow abstraction used in *TexeraAgent* also improves the performance of the agent. The extracted flow context and data context represent observations in a more concise format compared to Script ReAct. This representation allows the agent to retain useful context over longer interactions and focus more on high-level reasoning. To evaluate this benefit, we compared *TexeraAgent* with a script-based ReAct baseline built on smolagents [15] using KramaBench [13], a data science benchmark that contains 104 tasks, with GPT-5-mini and GPT-5.2. As shown in Table 1, *TexeraAgent* achieved higher accuracy at lower cost on two popular LLM models. For example, when using GPT-5-mini, Script ReAct achieved an accuracy of 63.5% with a cost of \$0.0204 per task, while *TexeraAgent* achieved a higher accuracy of 76.9% with a lower cost of \$0.0168 per task.

Table 1: Comparison results on KramaBench [13]

Agent	LLM			
	GPT-5-mini		GPT-5.2	
	Accuracy (%)	Cost/task (\$)	Accuracy (%)	Cost/task (\$)
Script Agent	63.5	0.0204	68.3	0.0964
<i>TexeraAgent</i>	76.9	0.0168	79.1	0.0843
Improvements	+13.4	-17.6%	+10.8	-12.6%

4 RELATED WORK

Script-based data science agents such as DA-Agent [8] and DS-Agent [6] generate and execute Python scripts in a ReAct loop. Because code serves as both the execution artifact and the agent context, these agents suffer from noisy contexts filled with low-level code details such as syntax, variables, and their values after execution. Multi-agent frameworks such as Data Interpreter [7] and DS-STAR [14] decompose tasks to use specialized sub-agents for better planning and error recovery, but their reasoning remains script-driven, inheriting the same context noise with additional multi-agent orchestration overhead. Both single-agent and multi-agent approaches provide limited or no interfaces for analysts, preventing effective human-agent collaboration on tasks requiring human feedback. LLM-powered systems such as SiriusBI [10] and DataChat [11] do offer interactive interfaces, but they internally still generate scripts as context and expose them through chat-based or notebook-like interfaces. Our work differs from these studies by using dataflow as the primary abstraction in human-agent collaboration, offering the aforementioned benefits.

ACKNOWLEDGMENTS

This work was supported by NIH NIDDK award 2U24DK097771-11 and a grant from SAP.

REFERENCES

- [1] Tyler Akidau, Robert Bradshaw, Craig Chambers, et al. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *VLDB* (2015).
- [2] Apache Texera. 2026. Apache Texera. <https://texera.apache.org/>.
- [3] Databricks. 2026. Use Genie Code for data science. <https://docs.databricks.com/aws/en/notebooks/ds-agent>. Accessed: 2026-03-27.
- [4] Ian Drosos, Advait Sarkar, Xiaotong Xu, Carina Negreanu, et al. 2024. “It’s like a rubber duck that talks back”: Understanding Generative AI-Assisted Data Analysis Workflows through a Participatory Prompting Study (*CHIWORK ’24*).
- [5] Ken Gu, Ruoxi Shang, Tim Althoff, et al. 2024. How Do Analysts Understand and Verify AI-Assisted Data Analyses? (*CHI ’24*).
- [6] Siyuan Guo, Cheng Deng, Ying Wen, et al. 2024. DS-Agent: Automated Data Science by Empowering Large Language Models with Case-Based Reasoning (*Proceedings of Machine Learning Research*).
- [7] Sirui Hong, Yizhang Lin, et al. 2025. Data Interpreter: An LLM Agent for Data Science. In *ACL*.
- [8] Yiming Huang, Jianwen Luo, Yan Yu, et al. 2024. DA-Code: Agent Data Science Code Generation Benchmark for Large Language Models.
- [9] Michael Isard, Mihai Badiu, Yuan Yu, et al. 2007. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks.
- [10] Jie Jiang, Haining Xie, et al. 2025. SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence. *Proc. VLDB Endow.* (2025).
- [11] Rogers Jeffrey Leo John, Dylan Bacon, Junda Chen, et al. 2023. DataChat: An Intuitive and Collaborative Data Analytics Platform (*SIGMOD ’23*). 203–215.
- [12] Majeed Kazemitabaar et al. 2024. Improving Steering and Verification in AI-Assisted Data Analysis with Interactive Task Decomposition (*UIST ’24*).
- [13] Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, et al. 2025. KramaBench: Evaluating End-to-End Data-Science Agents.
- [14] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, et al. 2026. DS-STAR: Data Science Agent for Solving Diverse Tasks across Heterogeneous Formats and Open-Ended Queries. *arXiv:2509.21825*.
- [15] Aymeric Roucher et al. 2025. smolagents: a smol library to build great agentic systems. <https://github.com/huggingface/smolagents>.
- [16] Zuoqi Wang, Yicong Huang, Shengquan Ni, Avinash Kumar, Sadeem Alsudais, Xiaozhen Liu, Xinyuan Lin, Yunyan Ding, and Chen Li. 2024. Texera: A System for Collaborative and Interactive Data Analytics Using Workflows. *VLDB* (2024).